

zadania_lab_13

Zadanie 1 — przeskalowanie wartości generatora

Polecenie:

Wykorzystaj znane generatory, np. `rand()`, do zwrócenia wartości z określonych przedziałów, przy założeniu, że `MAX` to największa wartość zwracana przez generator:

a)

$$(int) \langle 0, MAX \rangle$$

b)

$$(int) \langle 0, max \rangle, \quad max < MAX$$

c)

$$(int) \langle min, max \rangle, \quad min < max < MAX$$

d)

$$(double) \langle 0, 1 \rangle$$

O co chodzi w zadaniu?

Zakładamy, że mamy generator, który zwraca liczby całkowite z zakresu:

$$X \in 0, 1, \dots, MAX$$

Czyli generator może zwrócić:

$$0$$

ale może też zwrócić największą wartość:

$$MAX$$

Naszym zadaniem jest przeskalowanie tej liczby tak, żeby dostać wynik z innego przedziału.

Podpunkt a) — przedział $(int) \langle 0, MAX \rangle$

W tym przypadku nie trzeba nic przeliczać, ponieważ funkcja `rand()` już zwraca liczbę z przedziału:

$$\langle 0, MAX \rangle$$

Czyli:

$$Y = X$$

W kodzie wystarczy zwrócić wynik funkcji `rand()`.

Podpunkt b) — przedział $(int)\langle 0, max \rangle$

Chcemy dostać liczbę całkowitą z przedziału:

$$0, 1, \dots, max$$

gdzie:

$$max < MAX$$

Ze slajdu korzystamy ze wzoru:

$$Y = \left\lfloor \frac{X}{MAX + 1} (max + 1) \right\rfloor$$

Dlaczego jest $MAX + 1$?

Ponieważ jeśli generator zwraca liczby:

$$0, 1, 2, \dots, MAX$$

to wszystkich możliwych wartości jest:

$$MAX + 1$$

Podpunkt c) — przedział $(int)\langle min, max \rangle$

Chcemy dostać liczbę całkowitą z przedziału:

$$min, min + 1, \dots, max$$

Ze slajdu korzystamy ze wzoru:

$$Y = min + \left\lfloor \frac{X}{MAX + 1} (max - min + 1) \right\rfloor$$

Najpierw losujemy wartość z przedziału od 0 do $max - min$, a potem przesuwamy ją o min .

Podpunkt d) — przedział $(double)[0, 1]$

W treści zadania pojawia się zapis:

$$\langle 0, 1 \rangle$$

na wykładzie przeskalowanie do rozkładu jednostajnego jest podane jako:

$$R = \frac{X}{MAX + 1}$$

Wtedy:

$$R \in [0, 1)$$

Czyli wynik może być równy 0, ale nie będzie równy dokładnie 1.

Jeżeli generator zwróci największą możliwą wartość:

$$X = MAX$$

to:

$$R = \frac{MAX}{MAX + 1} < 1$$

Dlatego w kodzie zgodnym z wykładem używamy:

```
return X / (MAX + 1)
```

PYTHON

Jeśli zakres ma być faktycznie $[0, 1]$ to:

$$R = \frac{MAX}{MAX} = 1$$

```
return X / (MAX)
```

PYTHON

Kod programu

zad 1

```
import random # importujemy moduł random, ponieważ potrzebujemy gotowego
generatora liczb losowych
import sys # importujemy moduł sys, żeby użyć sys.maxsize jako dużej wartości MAX
```

```
MAX = sys.maxsize # ustalamy MAX jako największą dużą liczbę całkowitą dostępną w
systemie
```

```
def rand(): # definiujemy funkcję podobną do rand()
    return random.randint(0, MAX) # zwracamy liczbę całkowitą z przedziału <0,
MAX>
```

a)

```
def losuj_0_MAX(): # funkcja realizuje podpunkt a)
    return rand() # zwracamy wartość z generatora, bo rand() już daje przedział
<0, MAX>
```

b)

```
def losuj_0_max(max_wartosc): # funkcja realizuje podpunkt b)
    X = rand() # losujemy liczbę X z przedziału <0, MAX>

    return (X * (max_wartosc + 1)) // (MAX + 1) # stosujemy wzór floor(X/(MAX+1)*
(max+1))
```

c)

```
def losuj_min_max(min_wartosc, max_wartosc): # funkcja realizuje podpunkt c)
    X = rand() # losujemy liczbę X z przedziału <0, MAX>

    return min_wartosc + (X * (max_wartosc - min_wartosc + 1)) // (MAX + 1) #
stosujemy wzór ze slajdu
```

d)

```
def losuj_0_1(): # funkcja realizuje podpunkt d)
    X = rand() # losujemy liczbę X z przedziału <0, MAX>

    return X / (MAX) # otrzymujemy liczbę z przedziału [0,1]

# ALTERNATYWNIIE
# return X / (MAX + 1) # otrzymujemy liczbę z przedziału [0,1)
```

Test programu

```
PYTHON
print("-----ZADANIE 1-----") # wypisujemy nagłówek zadania

print("a) <0, MAX>:", losuj_0_MAX()) # testujemy losowanie z przedziału <0, MAX>
print("b) <0, max>:", losuj_0_max(10)) # testujemy losowanie z przedziału <0, 10>
print("c) <min, max>:", losuj_min_max(5, 15)) # testujemy losowanie z przedziału <5, 15>
print("d) <0, 1>:", losuj_0_1()) # testujemy losowanie liczby rzeczywistej z przedziału [0,1]
```

Wnioski

W zadaniu 1 najważniejsze jest poprawne przeskalowanie liczby X , którą zwraca generator.

Dla przedziałów całkowitych używamy wzorów:

$$Y = \left\lfloor \frac{X}{MAX + 1} (max + 1) \right\rfloor$$

oraz:

$$Y = min + \left\lfloor \frac{X}{MAX + 1} (max - min + 1) \right\rfloor$$

Dla przedziału rzeczywistego zgodnie z wykładem używamy:

$$R = \frac{X}{MAX + 1}$$

Wtedy:

$$R \in [0, 1)$$

Nie używamy prostego:

```
PYTHON
X % (max + 1)
```

ponieważ może ono powodować nierównomierny rozkład wyników, jeśli liczba możliwych wartości generatora nie dzieli się przez $max + 1$.

Zadanie 2 — addytywny generator LCG

Polecenie:

Zaimplementuj własny generator liczb pseudolosowych addytywny LCG oparty na wzorze:

$$X_{n+1} = aX_n + c \mod M$$

Przetestuj uzyskany generator następująco:

- utwórz zbiór punktów postaci:

$$(X_0, X_1), (X_2, X_3), \dots, (X_i, X_{i+1}), (X_{i+2}, X_{i+3}), \dots$$

- zwizualizuj tak utworzony zbiór punktów, np. jako plik SVG za pomocą:
https://www.w3schools.com/graphics/svg_circle.asp.

O co chodzi w zadaniu?

Generator LCG, czyli liniowy generator kongruencyjny, tworzy kolejne liczby na podstawie poprzedniej liczby.

Wzór ma postać:

$$X_{n+1} = (aX_n + c) \mod M$$

gdzie:

- X_n — aktualna wartość ciągu,
- X_{n+1} — następna wartość ciągu,
- a — mnożnik,
- c — przyrost,
- M — moduł,
- X_0 — ziarno, czyli wartość początkowa. (najlepiej wybrać $0 \leq X_0 < M$)

Dlaczego trzeba ustawić konkretne parametry?

Parametry a , c , M i X_0 nie powinny być wybrane całkiem przypadkowo.

Jeśli wybierzemy je źle, generator może mieć bardzo krótki okres, czyli ciąg szybko zacznie się powtarzać.

Dla generatora mieszanego:

$$X_{n+1} = (aX_n + c) \mod M$$

pełny okres można uzyskać, gdy spełnione są warunki Hullu–Dobella:

$$\text{nwd}(c, M) = 1$$

2. $a - 1$ jest podzielne przez każdy czynnik pierwszy liczby M ,
 3. jeśli M jest podzielne przez 4, to $a - 1$ też jest podzielne przez 4.
-

Parametry użyte w programie

W programie używamy typowych parametrów generatora ANSIC:

$$a = 1103515245$$

$$c = 12345$$

$$M = 2^{31}$$

$$X_0 = 7$$

Są one lepsze niż przypadkowe wartości typu:

```
a = 100000  
c = 12345  
M = 1515151
```

PYTHON

bo parametry z ANSIC są znanym zestawem dla generatora LCG.

Tworzenie punktów

Z wygenerowanego ciągu tworzymy punkty:

$$(X_0, X_1), (X_2, X_3), (X_4, X_5), \dots$$

Czyli bierzemy liczby parami:

```
(liczby[0], liczby[1])  
(liczby[2], liczby[3])  
(liczby[4], liczby[5])
```

PYTHON

itd.

Do rysowania punktów w pliku SVG trzeba je przeskalować, bo wartości LCG są z przedziału:

$$0, 1, \dots, M - 1$$

Dlatego współrzędne przeliczamy na rozmiar obrazka.

Kod programu

zad 2

```
def generator_LCG(a, c, X0, M, ile): # definiujemy funkcję generatora LCG | (ile
to jest ile liczb ma wygenerować)
    liczby = [] # tworzymy pustą listę na wygenerowane liczby
    X = X0 # ustawiamy wartość początkową generatora, czyli ziarno

    for i in range(ile): # wykonujemy pętlę tyle razy, ile liczb chcemy
wygenerować
        liczby.append(X) # zapisujemy aktualną wartość X do listy
        X = (a * X + c) % M # obliczamy następną wartość zgodnie ze wzorem LCG

    return liczby # zwracamy listę wygenerowanych liczb

def nwd(a, b): # funkcja oblicza największy wspólny dzielnik liczb a i b
    a = abs(a) # zamieniamy a na wartość dodatnią
    b = abs(b) # zamieniamy b na wartość dodatnią

    while b != 0: # wykonujemy pętlę, dopóki b nie jest równe 0
        reszta = a % b # obliczamy resztę z dzielenia a przez b
        a = b # przesuwamy b na miejsce a
        b = reszta # reszta z dzielenia staje się nowym b

    return a # gdy b = 0, aktualne a jest największym wspólnym dzielnikiem

def utworz_punkty(liczby): # definiujemy funkcję tworzącą punkty z kolejnych
wartości ciągu
    punkty = [] # tworzymy pustą listę punktów

    for i in range(0, len(liczby) - 1, 2): # przechodzimy po liście co dwa
elementy
        punkty.append((liczby[i], liczby[i + 1])) # tworzymy punkt (X_i, X_{i+1})

    return punkty # zwracamy listę punktów

def zapisz_svg(punkty, M, nazwa_pliku): # definiujemy funkcję zapisującą punkty
do pliku SVG
    szerokosc = 500 # ustalamy szerokość obrazka
    wysokosc = 500 # ustalamy wysokość obrazka
    margines = 20 # ustalamy margines od krawędzi

    svg = f'<svg width="{szerokosc}" height="{wysokosc}" viewBox="0 0 {szerokosc}
{wysokosc}" xmlns="http://www.w3.org/2000/svg">\n' # rozpoczynamy plik SVG
    svg += '<rect width="100%" height="100%" fill="white"/>\n' # dodajemy białe
tło
    svg += f'<rect x="{margines}" y="{margines}" width="{szerokosc - 2*margines}"
height="{wysokosc - 2*margines}" fill="none" stroke="black"/>\n' # dodajemy ramkę
wykresu

    for x, y in punkty: # przechodzimy po wszystkich punktach
        x_svg = margines + (x / M) * (szerokosc - 2 * margines) # skalujemy
```

```
współrzedną x do szerokości obrazka
    y_svg = wysokosc - margines - (y / M) * (wysokosc - 2 * margines) #
skalujemy współrzedną y i odwracamy oś pionową

    svg += f'<circle cx="{x_svg}" cy="{y_svg}" r="3" fill="blue"/>\n' #
dodajemy punkt jako niebieskie kółko

svg += '</svg>' # kończymy plik SVG

with open(nazwa_pliku, "w", encoding="utf-8") as plik: # otwieramy plik do
zapisu
    plik.write(svg) # zapisujemy tekst SVG do pliku
```

Test programu

PYTHON

```
print("-----ZADANIE 2-----")

a = 1103515245 # mnożnik generatora LCG
c = 12345 # przyrost generatora LCG
M = 2**31 # moduł generatora LCG
X0 = 7 # ziarno generatora

print("Parametry generatora LCG:") # wypisujemy opis parametrów
print("a =", a) # wypisujemy a
print("c =", c) # wypisujemy c
print("M =", M) # wypisujemy M
print("X0 =", X0) # wypisujemy ziarno

print("\nSprawdzenie podstawowego warunku:") # wypisujemy nagłówek sprawdzenia
print("moja funkcja z wyżej: nwd(c, M) =", nwd(c, M)) # sprawdzamy, czy c i M są
względnie pierwsze
# print("nwd(c, M) =", math.gcd(c, M)) # funkcja z biblioteki - wymaga import math

liczby = generator_LCG(a, c, X0, M, 1000) # generujemy 1000 liczb pseudolosowych
punkty = utworz_punkty(liczby) # tworzymy punkty z kolejnych wartości ciągu

print("\nPierwsze 20 liczb:") # wypisujemy nagłówek
print(liczby[:20]) # wypisujemy pierwsze 20 liczb

# Wypisanie wszystkich liczb
# print("Wygenerowane liczby:")
# for i in range(len(liczby)):
# print("X_", i, "=", liczby[i])

print("\nPierwsze 10 punktów:") # wypisujemy nagłówek
print(punkty[:10]) # wypisujemy pierwsze 10 punktów

# Wypisanie wszystkich punktów
# print("\nWygenerowane punkty:")
# for i in range(len(punkty)):
# print("P_", i, "=", punkty[i])

zapisz_svg(punkty, M, "punkty_LCG.svg") # zapisujemy punkty do pliku SVG

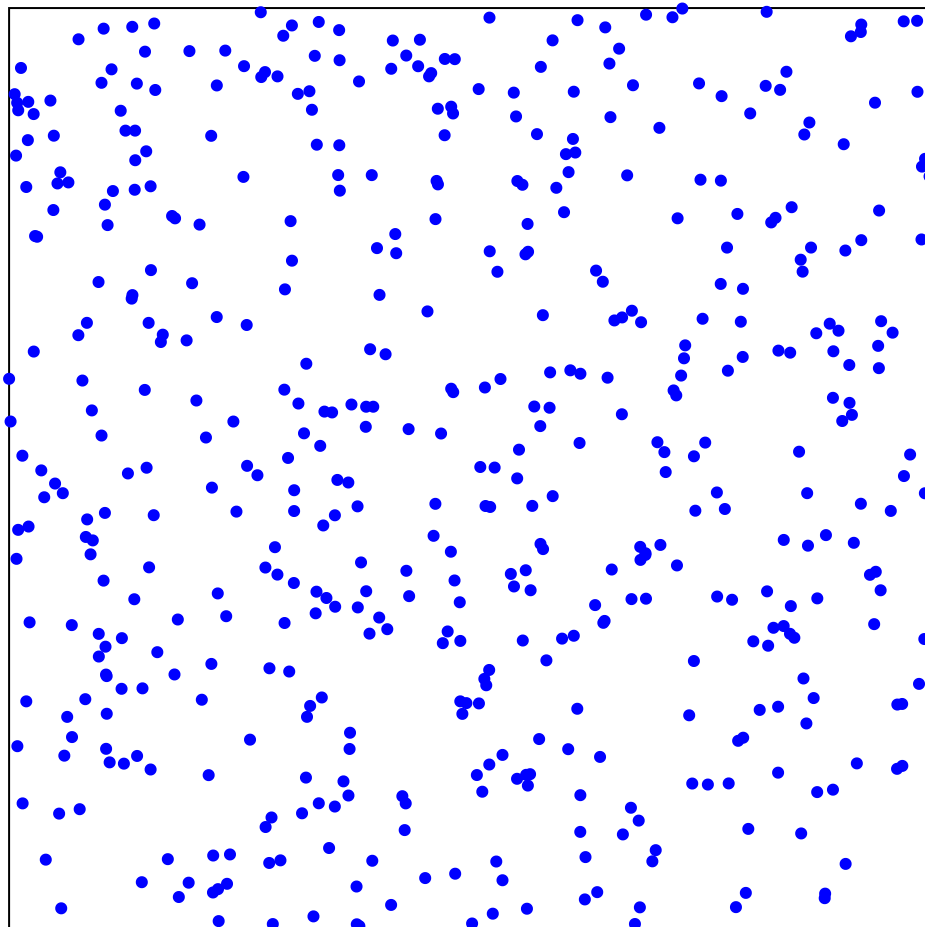
print("\nZapisano plik punkty_LCG.svg") # informujemy o zapisaniu pliku
```

Co oznacza plik SVG?

Plik:

zawiera wizualizację punktów:

$$(X_0, X_1), (X_2, X_3), (X_4, X_5), \dots$$



Jeżeli punkty układają się w wyraźne linie albo regularne pasy, generator może mieć słabe własności statystyczne.

W przypadku generatorów LCG takie zależności mogą być widoczne, ponieważ generator jest deterministyczny i liniowy.

Wnioski

W zadaniu 2 zaimplementowano addytywny generator LCG:

$$X_{n+1} = (aX_n + c) \mod M$$

Użyto parametrów:

$$a = 1103515245$$

$$c = 12345$$

$$M = 2^{31}$$

$$X_0 = 7$$

Następnie z wygenerowanych liczb utworzono punkty:

$$(X_0, X_1), (X_2, X_3), (X_4, X_5), \dots$$

i zapisano je do pliku SVG.

Dzięki temu można wizualnie sprawdzić, czy punkty wyglądają losowo, czy tworzą regularne wzory.

Zadanie 3 — generator LFG

Polecenie:

Zaimplementuj własny generator liczby pseudolosowych LFG oparty na wzorze:

$$X_n = X_{n-q} + X_{n-p} \mod M$$

gdzie:

$$1 \leq q \leq p \leq M$$

O co chodzi w zadaniu?

Generator LFG, czyli lagged Fibonacci generator, jest uogólnieniem generatora Fibonacciego.

Zamiast korzystać tylko z dwóch poprzednich wartości, generator korzysta z wartości oddalonych o pewne opóźnienia.

Na wykładzie wzór ma postać:

$$X_n = (X_{n-p} + X_{n-q}) \mod m$$

gdzie:

$$n \geq p$$

oraz:

$$p > q \geq 1$$

Liczby p i q oznaczają opóźnienia.

Generator opóźniony można zmodyfikować, zmieniając operację wykonywaną na poprzednich wyrazach ciągu:

$$X_n = (X_{n-p} \diamond X_{n-q}) \mod m$$

gdzie symbol $\diamond$ oznacza wybraną operację, np.:

$$\diamond \in \{+, -, \cdot, \oplus\}$$

czyli dodawanie, odejmowanie, mnożenie albo XOR.

Przykładowe wersje:

Dodawanie:

$$X_n = (X_{n-p} + X_{n-q}) \bmod m$$

Odejmowanie:

$$X_n = (X_{n-p} - X_{n-q}) \bmod m$$

Mnożenie:

$$X_n = (X_{n-p} \cdot X_{n-q}) \bmod m$$

XOR:

$$X_n = (X_{n-p} \oplus X_{n-q}) \bmod m$$

Wartości początkowe

Do uruchomienia generatora LFG trzeba podać p wartości początkowych:

$$X_0, X_1, \dots, X_{p-1}$$

Jeżeli:

$$p = 3$$

to trzeba podać trzy wartości początkowe:

$$X_0, X_1, X_2$$

Wartości początkowe nie powinny być samymi zerami, bo wtedy generator mógłby dawać ciąg trywialny, czyli same zera.

Parametry zgodne z przykładem z wykładu

W programie używamy przykładu z wykładu:

$$M = 17$$

$$p = 3$$

$$q = 1$$

Wartości początkowe:

$$X_0 = 7$$

$$X_1 = 16$$

$$X_2 = 5$$

- Wartości początkowe muszą należeć do zakresu:

$$0 \leq X_i < M$$

Sprawdzamy warunek:

$$p > q \geq 1$$

czyli:

$$3 > 1 \geq 1$$

Warunek jest spełniony.

Pierwsze obliczenia

Dla:

$$X_n = (X_{n-p} + X_{n-q}) \mod M$$

oraz:

$$M = 17, \quad p = 3, \quad q = 1$$

mamy:

$$X_3 = (X_{3-3} + X_{3-1}) \mod 17$$

czyli:

$$X_3 = (X_0 + X_2) \mod 17$$

Podstawiamy:

$$X_3 = (7 + 5) \mod 17 = 12$$

Następnie:

$$X_4 = (X_1 + X_3) \mod 17$$

$$X_4 = (16 + 12) \mod 17 = 28 \mod 17 = 11$$

Następnie:

$$X_5 = (X_2 + X_4) \mod 17$$

$$X_5 = (5 + 11) \mod 17 = 16$$

Początek ciągu to:

$$7, 16, 5, 12, 11, 16, \dots$$

Kod programu

zad 3

```

def generator_LFG_dodawanie(p, q, M, poczatkowe, ile): # generator LFG z
    dodawaniem
    if not (p > q >= 1): # sprawdzamy warunek  $p > q \geq 1$ 
        raise ValueError("Musi być spełniony warunek  $p > q \geq 1$ .")

    if p > M: # sprawdzamy dodatkowy warunek z wykładu
        raise ValueError("Musi być spełniony warunek  $p \leq M$ .")

    if len(poczatkowe) < p: # sprawdzamy, czy podano co najmniej p wartości
    początkowych
        raise ValueError("Trzeba podać co najmniej p wartości początkowych.")

    if all(wartosc == 0 for wartosc in poczatkowe): # sprawdzamy, czy wartości
    początkowe nie są samymi zerami
        raise ValueError("Wartości początkowe nie mogą być samymi zerami.")

    liczby = poczatkowe.copy() # kopiujemy wartości początkowe

    for n in range(p, ile): # generujemy kolejne wartości ciągu
        Xn = (liczby[n - p] + liczby[n - q]) % M # wzór:  $X_n = (X_{n-p} + X_{n-q}) \bmod M$ 
        liczby.append(Xn) # dodajemy nową wartość

    return liczby # zwracamy cały ciąg

def generator_LFG_wybor_operacji(p, q, M, X_poczatkowe, ile, operacja): #
    generator LFG z wyborem operacji
    if not (p > q >= 1): # sprawdzamy warunek  $p > q \geq 1$ 
        raise ValueError("Musi być spełniony warunek  $p > q \geq 1$ .")

    if p > M: # sprawdzamy dodatkowy warunek  $p \leq M$ 
        raise ValueError("Musi być spełniony warunek  $p \leq M$ .")

    if len(X_poczatkowe) < p: # sprawdzamy, czy podano co najmniej p wartości
    początkowych
        raise ValueError("Trzeba podać co najmniej p wartości początkowych.")

    if all(wartosc == 0 for wartosc in X_poczatkowe): # sprawdzamy, czy wartości
    początkowe nie są samymi zerami
        raise ValueError("Wartości początkowe nie mogą być samymi zerami.")

    liczby = X_poczatkowe.copy() # kopiujemy wartości początkowe

    for n in range(p, ile): # generujemy kolejne wartości ciągu
        a = liczby[n - p] # pobieramy  $X_{n-p}$ 
        b = liczby[n - q] # pobieramy  $X_{n-q}$ 

        if operacja == "dodawanie": # wersja z dodawaniem
            nowy = (a + b) % M

```

```

elif operacja == "odejmowanie": # wersja z odejmowaniem
    nowy = (a - b) % M

elif operacja == "mnozenie": # wersja z mnożeniem
    nowy = (a * b) % M

elif operacja == "xor": # wersja z operacją XOR
    nowy = (a ^ b) % M

else: # jeśli wpisano nieznaną operację
    raise ValueError("Nieznana operacja.")

liczby.append(nowy) # dopisujemy nową wartość do ciągu

return liczby # zwracamy wygenerowany ciąg

```

Test programu

PYTHON

```

print("-----ZADANIE 3-----") # wypisujemy nagłówek
zadania 3

M = 17 # moduł generatora zgodny z przykładem z wykładu
p = 3 # pierwsze opóźnienie
q = 1 # drugie opóźnienie

początkowe = [7, 16, 5] # wartości początkowe X0, X1, X2

liczby = generator_LFG(p, q, M, początkowe, 12) # generujemy 12 wartości ciągu
liczby2 = generator_LFG_wybor_operacji(p, q, M, początkowe, 12, "dodawanie") #
wersja z wyborem operacji

print("Parametry generatora LFG:") # wypisujemy opis parametrów
print("M =", M) # wypisujemy moduł
print("p =", p) # wypisujemy p
print("q =", q) # wypisujemy q
print("Wartości początkowe:", początkowe) # wypisujemy wartości początkowe

print("\nWygenerowany ciąg:") # wypisujemy nagłówek wyniku
print(liczby) # wypisujemy wygenerowany ciąg
print(liczby2) # wypisujemy drugi wygenerowany ciąg

```

Oczekiwany wynik

Dla parametrów z wykładu:

$$M = 17, \quad p = 3, \quad q = 1$$

oraz wartości początkowych:

$$7, 16, 5$$

ciąg powinien zacząć się tak:

```
[7, 16, 5, 12, 11, 16, 11, 5, 4, 15, 3, 7]
```

Jest to zgodne z obliczeniami:

$$X_3 = (7 + 5) \mod 17 = 12$$

$$X_4 = (16 + 12) \mod 17 = 11$$

$$X_5 = (5 + 11) \mod 17 = 16$$

Wnioski

W zadaniu 3 zaimplementowano generator LFG:

$$X_n = (X_{n-p} + X_{n-q}) \mod M$$

Generator wymaga podania:

- modułu (M),
- opóźnień (p) i (q),
- (p) wartości początkowych.

Wartości początkowe są konieczne, ponieważ bez nich nie da się obliczyć pierwszych wyrazów ciągu.

Dla przykładu z wykładu:

$$M = 17, \quad p = 3, \quad q = 1$$

oraz:

$$X_0 = 7, \quad X_1 = 16, \quad X_2 = 5$$

otrzymujemy ciąg:

$$7, 16, 5, 12, 11, 16, 11, 5, 4, 15, 3, 7, \dots$$

Generator LFG zmniejsza proste zależności między kolejnymi wyrazami w porównaniu z podstawowym generatorem Fibonacciego, ale nadal jest generatorem deterministycznym.